Magicdog-Y1 SDK Development Documentation

发布 *1.2.3-doc2*

MagicLab

2026 年 06 月 01 日

About MagicDog-Y1:

CHAPTER 1

About MagicDog-Y1

MagicDog-Y1 is a quadruped robot platform in the MagicDog family. The Y1 SDK provides APIs for robot connection, state monitoring, high-level motion control, motion mode switching, joystick control, and SLAM navigation.

This documentation currently opens the main controller, high-level motion, SLAM navigation, and state monitoring APIs. Sensor, audio, and low-level motion documentation is under development and is not public yet.

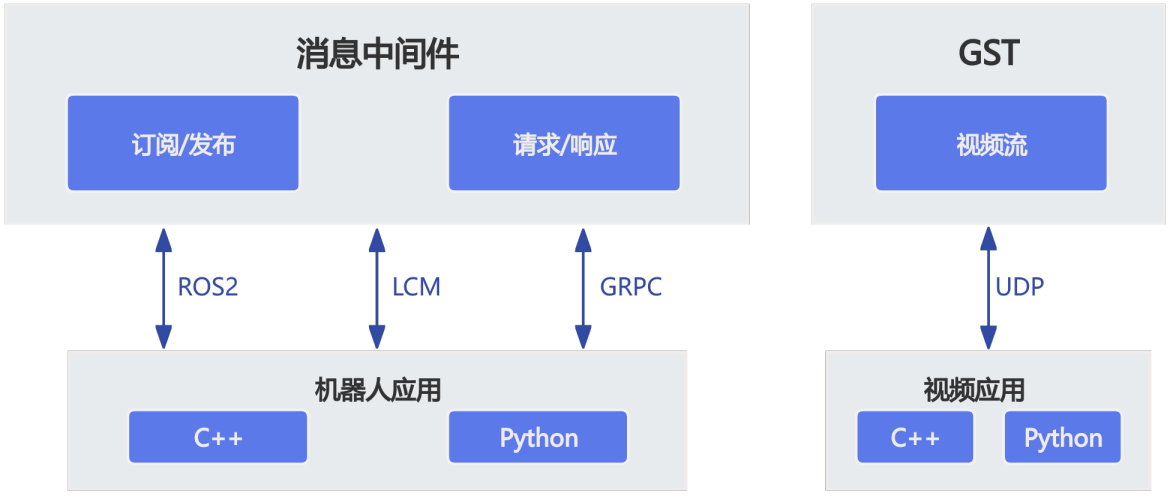
1.1 What is included

- SDK overview: communication model, architecture, service overview, and quick start.
- C++ API: main controller, high-level motion, SLAM navigation, and monitor APIs.
- Python API: Python bindings for the opened APIs.
- Examples: examples for the opened capabilities.

CHAPTER 2

SDK Overview

MagicDog-Y1 uses LCM, gRPC, and ROS 2 as communication middleware. The SDK wraps request-response calls and topic-based data exchange behind function-style C++ and Python APIs.



The current public documentation focuses on robot connection, high-level motion control, SLAM navigation, and state monitoring. Sensor, audio, and low-level motion documentation is under development.

2.1 SDK repositories

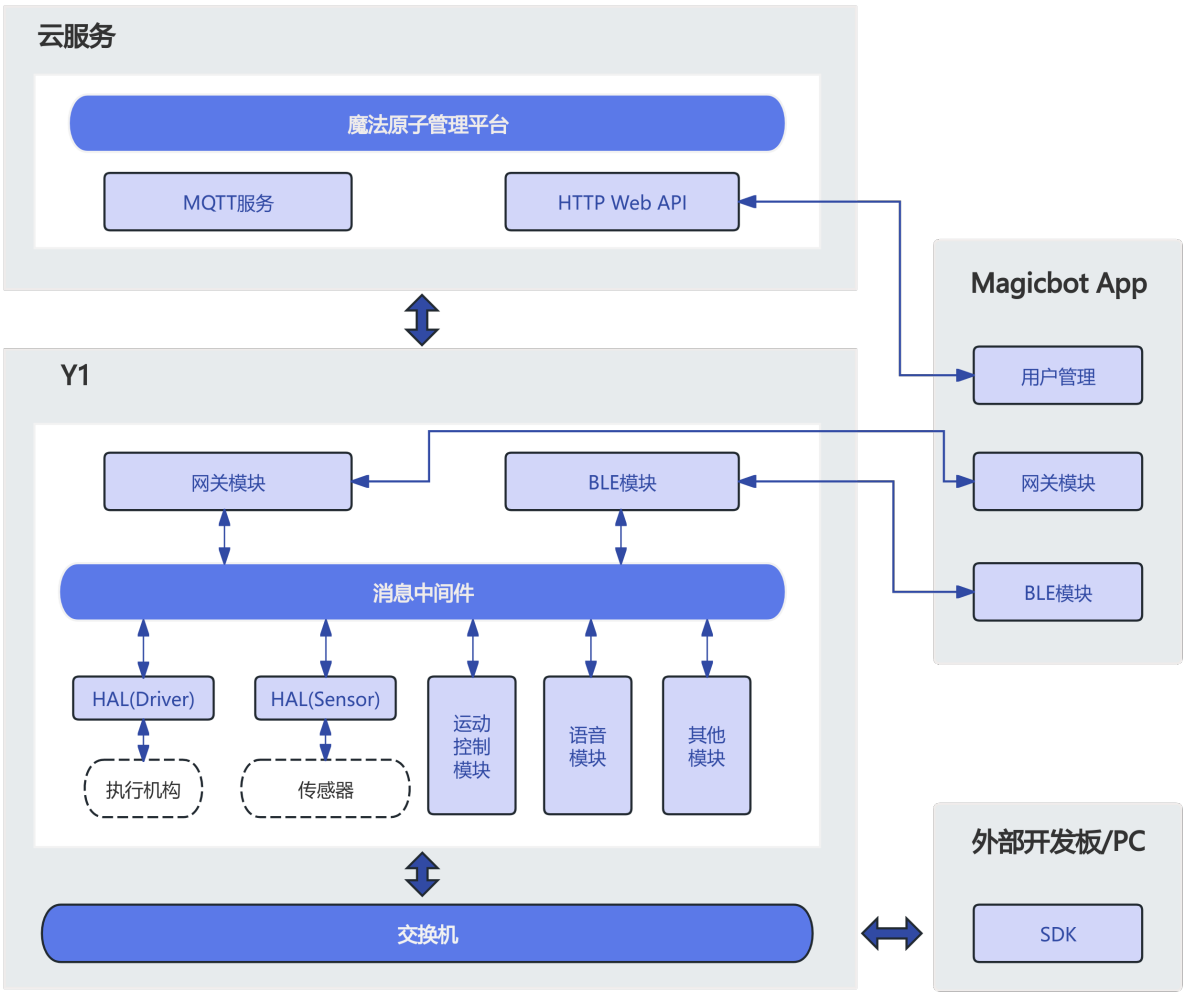
- magicbot-y1_sdk
- magicbot-y1_description

Some API and type descriptions on the documentation site may lag behind the GitHub repository. Use the SDK repository as the source of truth.

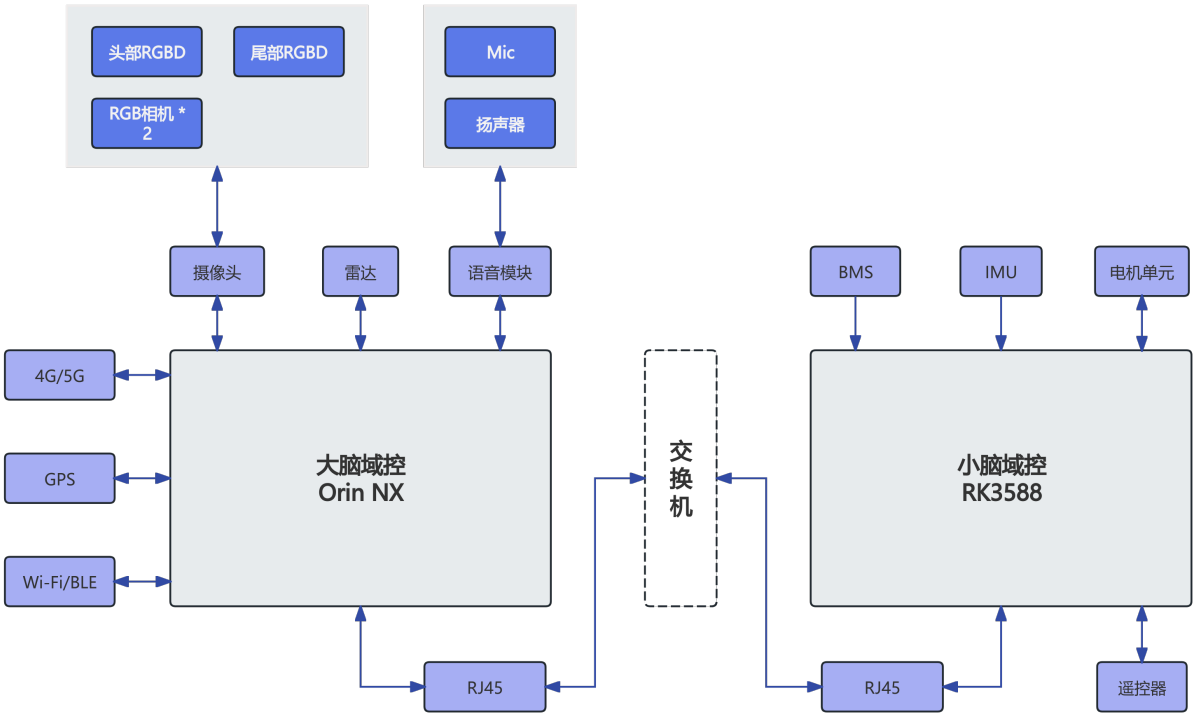
CHAPTER 3

SDK Architecture

The Y1 SDK exposes C++ and Python APIs at the application layer, and wraps robot services, communication middleware, and robot state data underneath. Application code can call the public controllers without handling low-level communication details directly.



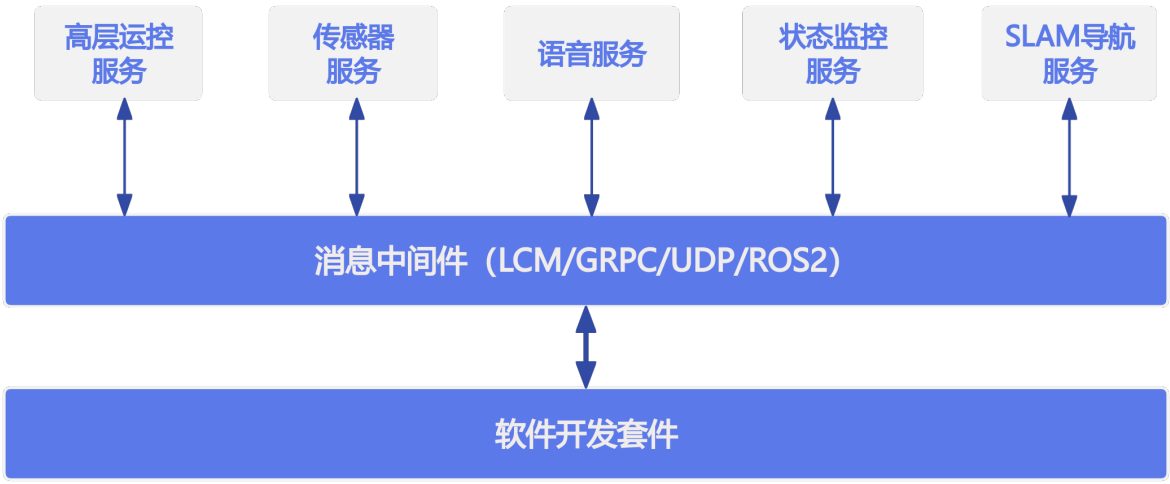
MagicDog-Y1 hardware consists of the robot body, onboard compute, sensors, and network communication links. Connect the development computer to the robot network over Ethernet before calling SDK APIs.


头部RGBD
尾部RGBD
RGB相机 * 2
Mic
扬声器
摄像头
雷达
语音模块
BMS
IMU
电机单元
4G/5G
GPS
Wi-Fi/BLE
大脑域控
Orin NX
交换机
小脑域控
RK3588
RJ45
RJ45
遥控器

CHAPTER 4

Service Overview

MagicDog-Y1 exposes services through LCM, gRPC, and ROS 2 middleware.



The current public service documentation covers high-level motion control, SLAM navigation, and state monitoring.

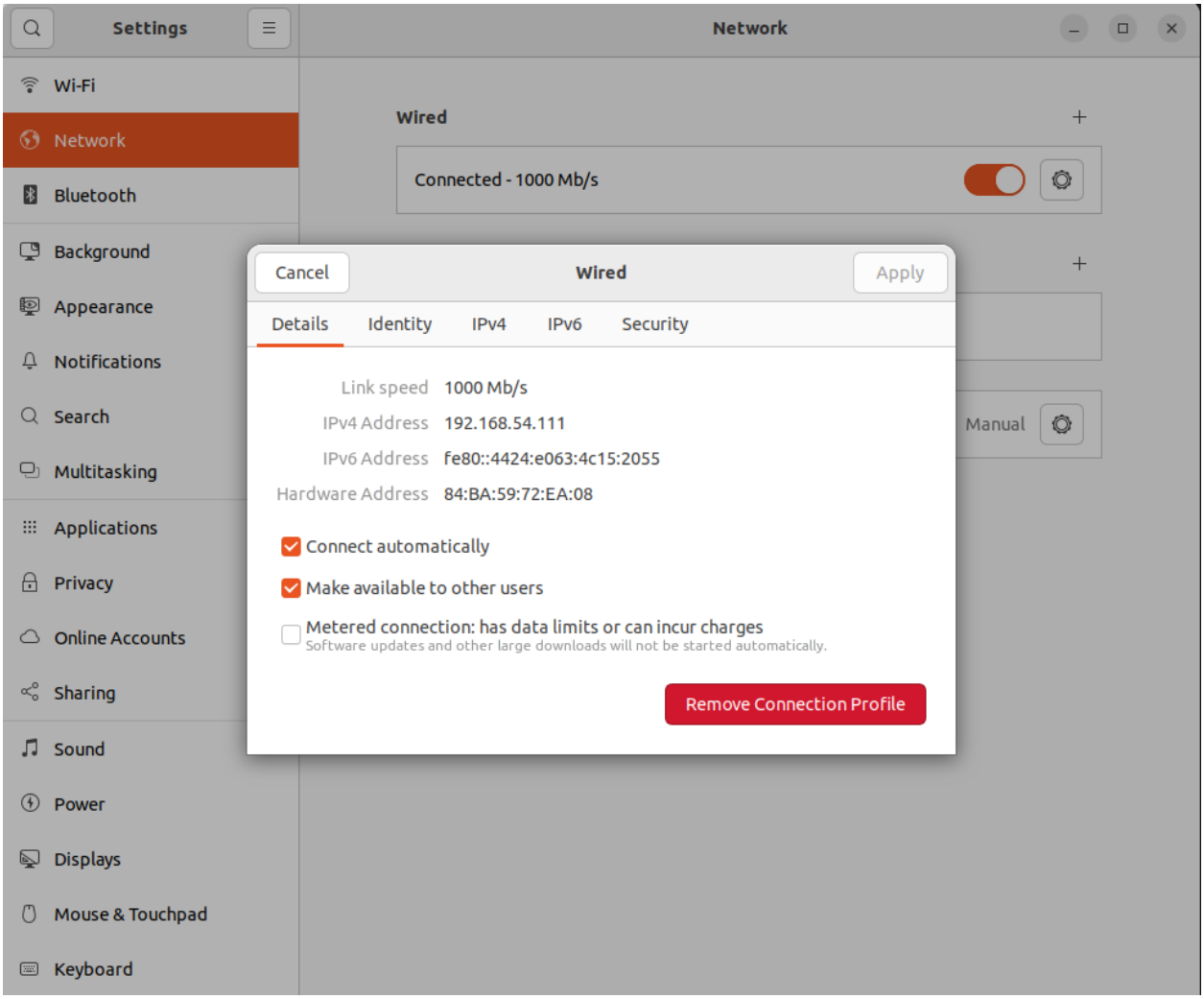
- **High-level motion service**: provides WalkId state switching, MotionId mode switching, posture control, and velocity control. It covers operations equivalent to remote controller control and uses gRPC for communication.
- **SLAM navigation service**: provides mapping, localization, navigation, map management, and odometry data access through RPC and topic subscription.
- **State monitor service**: provides robot hardware/software fault information and BMS battery status through gRPC.

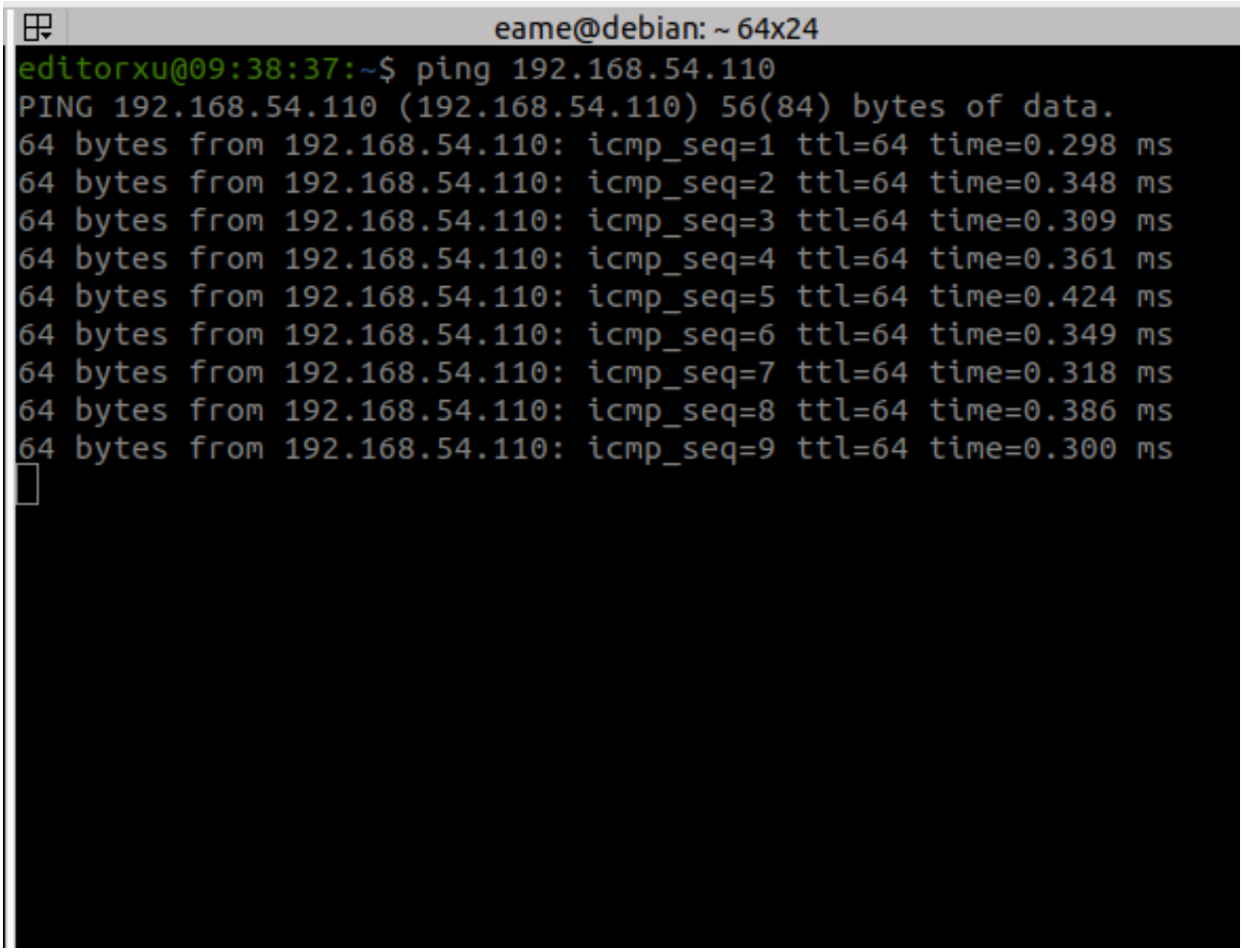
- **Low-level motion service**: under development and not public yet.
- **Audio service**: under development and not public yet.
- **Sensor service**: under development and not public yet.

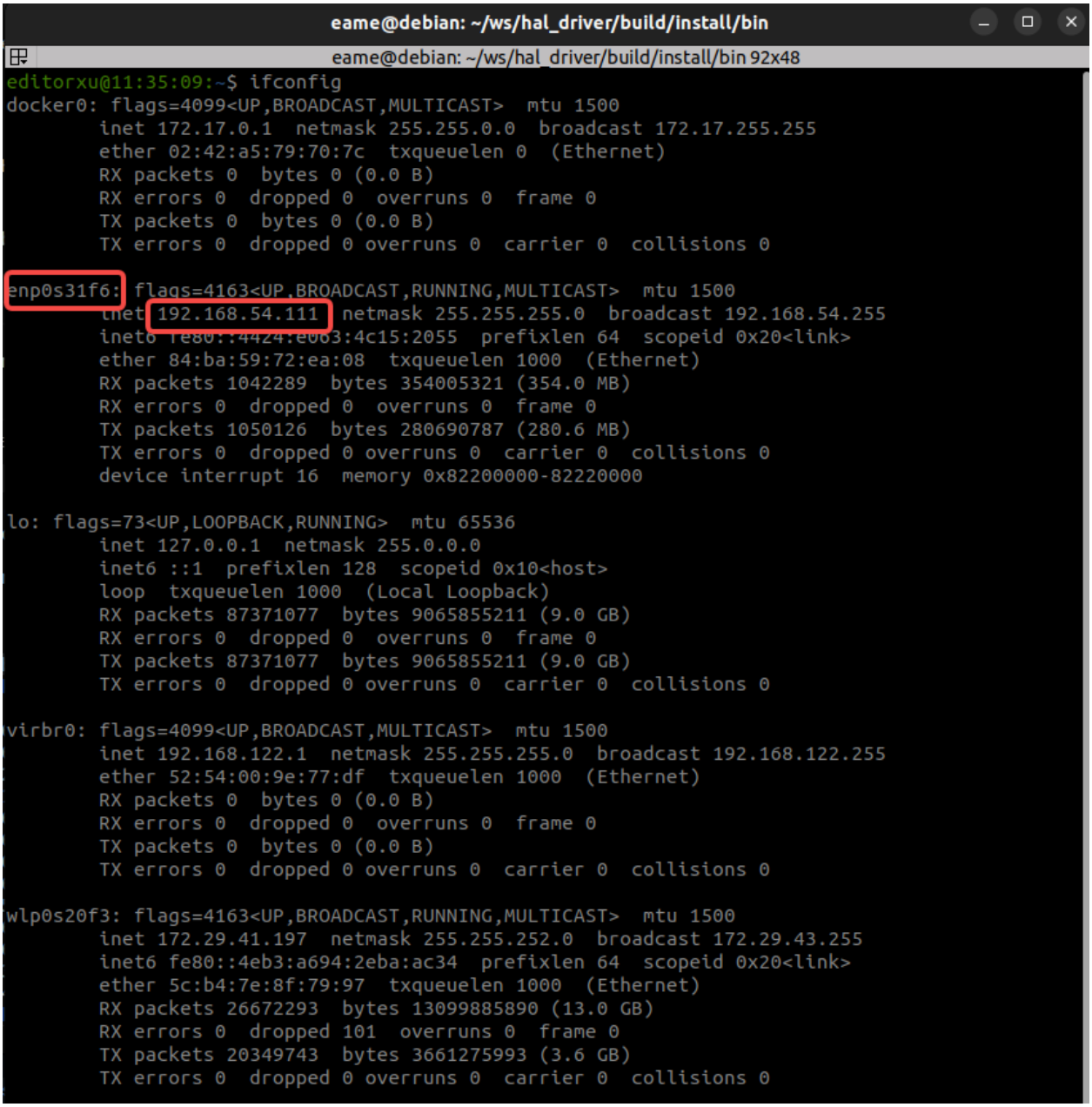
CHAPTER 5

Quick Start

Install and build the SDK according to the project README. Before connecting to the robot, make sure the development machine and robot are on the expected network.


Settings
Network
Wi-Fi
Network
Bluetooth
Background
Appearance
Notifications
Search
Multitasking
Applications
Privacy
Online Accounts
Sharing
Sound
Power
Displays
Mouse & Touchpad
Keyboard
Wired
Connected - 1000 Mb/s
Manual
Cancel
Wired
Apply
Details
Identity
IPv4
IPv6
Security
Link speed 1000 Mb/s
IPv4 Address 192.168.54.111
IPv6 Address fe80::4424:e063:4c15:2055
Hardware Address 84:BA:59:72:EA:08
Connect automatically
Make available to other users
Metered connection: has data limits or can incur charges
Software updates and other large downloads will not be started automatically.
Remove Connection Profile


eame@debian: ~ 64x24
editorxu@09:38:37:~$ ping 192.168.54.110
PING 192.168.54.110 (192.168.54.110) 56(84) bytes of data.
64 bytes from 192.168.54.110: icmp_seq=1 ttl=64 time=0.298 ms
64 bytes from 192.168.54.110: icmp_seq=2 ttl=64 time=0.348 ms
64 bytes from 192.168.54.110: icmp_seq=3 ttl=64 time=0.309 ms
64 bytes from 192.168.54.110: icmp_seq=4 ttl=64 time=0.361 ms
64 bytes from 192.168.54.110: icmp_seq=5 ttl=64 time=0.424 ms
64 bytes from 192.168.54.110: icmp_seq=6 ttl=64 time=0.349 ms
64 bytes from 192.168.54.110: icmp_seq=7 ttl=64 time=0.318 ms
64 bytes from 192.168.54.110: icmp_seq=8 ttl=64 time=0.386 ms
64 bytes from 192.168.54.110: icmp_seq=9 ttl=64 time=0.300 ms


eame@debian: ~/ws/hal_driver/build/install/bin
eame@debian: ~/ws/hal_driver/build/install/bin 92x48
editorxu@11:35:09:~$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
ether 02:42:a5:79:70:7c txqueuelen 0 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
enp0s31f6: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.54.111 netmask 255.255.255.0 broadcast 192.168.54.255
ether 84:ba:59:72:ea:08 txqueuelen 1000 (Ethernet)
RX packets 1042289 bytes 354005321 (354.0 MB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 1050126 bytes 280690787 (280.6 MB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
device interrupt 16 memory 0x82200000-82220000
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 87371077 bytes 9065855211 (9.0 GB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 87371077 bytes 9065855211 (9.0 GB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
virbr0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
inet 192.168.122.1 netmask 255.255.255.0 broadcast 192.168.122.255
ether 52:54:00:9e:77:df txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
wlp0s20f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 172.29.41.197 netmask 255.255.252.0 broadcast 172.29.43.255
inet6 fe80::4eb3:a694:2eba:ac34 prefixlen 64 scopeid 0x20<link>
ether 5c:b4:7e:8f:79:97 txqueuelen 1000 (Ethernet)
RX packets 26672293 bytes 13099885890 (13.0 GB)
RX errors 0 dropped 101 overruns 0 frame 0
TX packets 20349743 bytes 3661275993 (3.6 GB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

CHAPTER 6

Robot Main Control Service

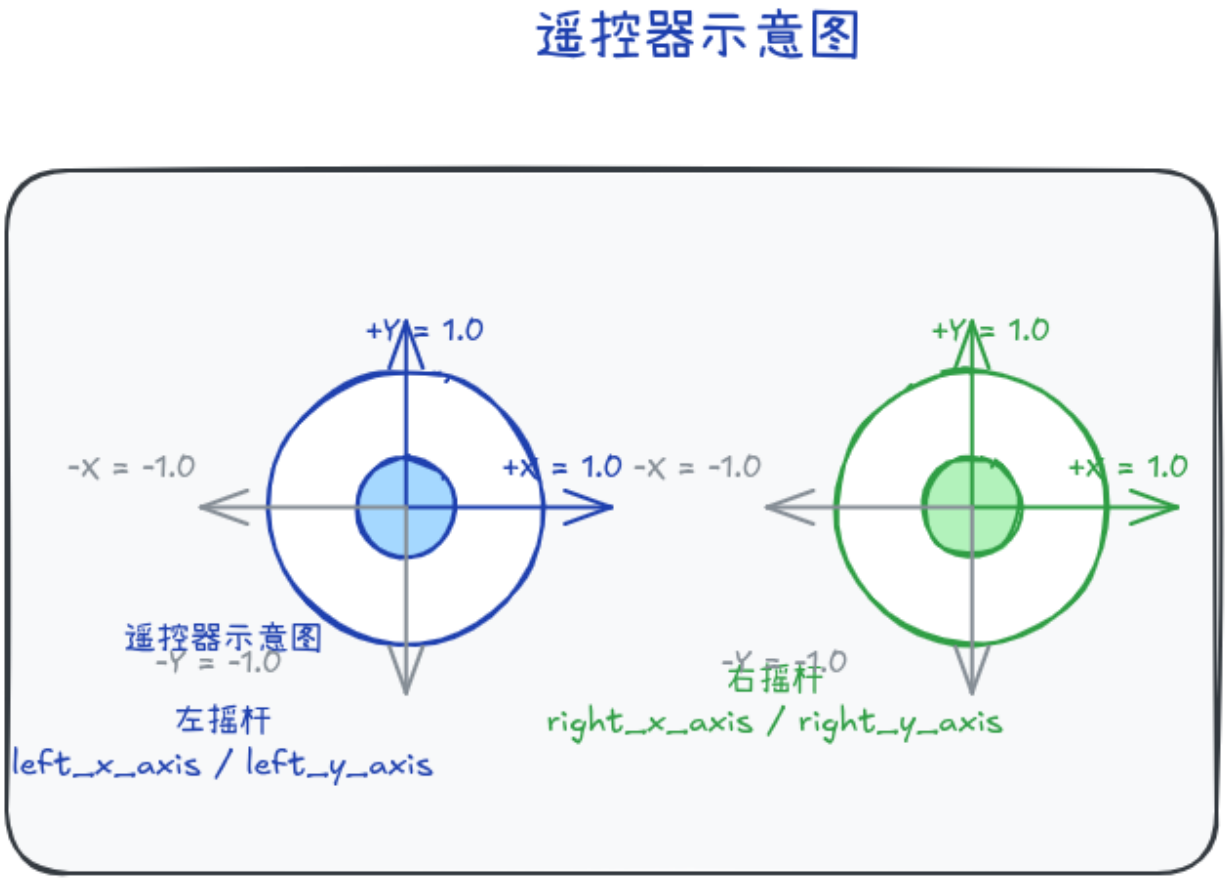
`MagicRobot` is the SDK entry point. Use it to initialize resources, connect to the robot, access opened controllers, disconnect, and shut down.

The detailed C++ API is being normalized for the public Y1 documentation. Use the SDK header files as the source of truth for function signatures.

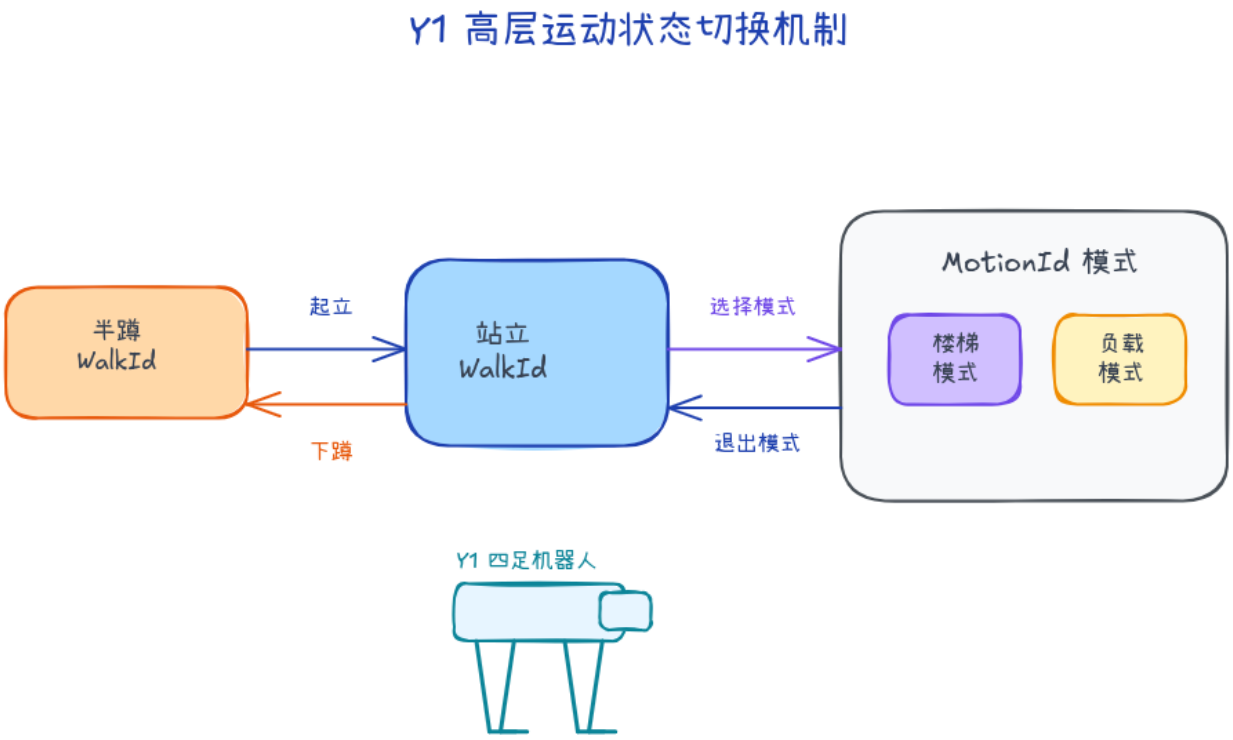
CHAPTER 7

High-Level Motion Control Service

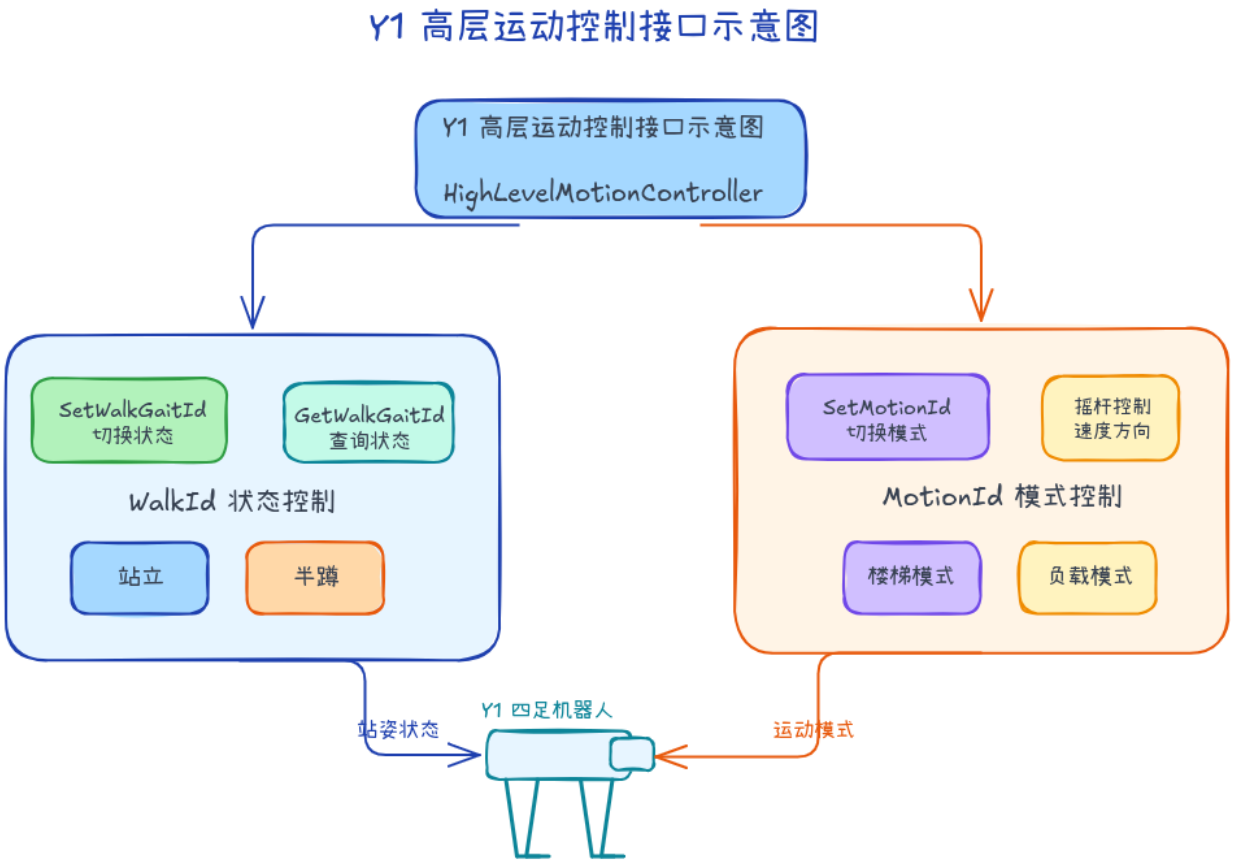
The high-level motion controller provides WalkId state control, MotionId mode control, and joystick control for MagicDog-Y1.



取值范围：[-1.0, 1.0]；上 / 右为正方向



高层运动只表达 Y1 四足运动状态：WalkId 控制站立/半蹲，MotionId 控制楼梯/负载模式



7.1 Motion state switching

CHAPTER 8

Low-Level Motion Control Service

This capability is under development. Public documentation is not available yet.

CHAPTER 9

Sensor Service

This capability is under development. Public documentation is not available yet.

CHAPTER 10

Audio Service

This capability is under development. Public documentation is not available yet.

CHAPTER 11

Monitor Service

The monitor service provides robot state query and subscription capabilities. The detailed API page is being normalized for the public Y1 documentation. Use the SDK header files as the source of truth for function signatures.

CHAPTER 12

SLAM Navigation Service

`SlamNavController` provides SLAM and navigation control for MagicDog-Y1. Use it to switch between mapping and localization, manage maps, set navigation targets, read navigation status, and subscribe to odometry.

12.1 Main workflow

1. Initialize the robot and get `SlamNavController`.
2. Call `Initialize()` before using SLAM or navigation APIs.
3. Use `ActivateSlamMode()` to enter mapping or localization mode.
4. Use mapping APIs to create and save maps, or load an existing map for localization.
5. Use `ActivateNavMode()` and `SetNavTarget()` to start navigation.
6. Use status and odometry APIs to monitor navigation.
7. Call `Shutdown()` when the controller is no longer needed.

12.2 Controller APIs

- `SlamNavController()`
- `~SlamNavController()`
- `bool Initialize()`
- `void Shutdown()`

- `Status ActivateSlamMode(SlamMode mode, std::string map_path = "", int timeout_ms = 5000)`
- `Status StartMapping(int timeout_ms = 5000)`
- `Status CancelMapping(int timeout_ms = 5000)`
- `Status SaveMap(std::string map_name, int timeout_ms = 5000)`
- `Status LoadMap(std::string map_path, int timeout_ms = 5000)`
- `Status DeleteMap(std::string map_path, int timeout_ms = 5000)`
- `std::string GetMapPath()`
- `AllMapInfo GetAllMapInfo()`
- `Status InitPose(Pose3DEuler pose, int timeout_ms = 5000)`
- `LocalizationInfo GetCurrentLocalizationInfo()`
- `Status ActivateNavMode(NavMode mode, int timeout_ms = 5000)`
- `Status SetNavTarget(NavTarget target, int timeout_ms = 5000)`
- `Status PauseNavTask(int timeout_ms = 5000)`
- `Status ResumeNavTask(int timeout_ms = 5000)`
- `Status CancelNavTask(int timeout_ms = 5000)`
- `NavStatus GetNavTaskStatus()`
- `Status OpenOdometryStream(int timeout_ms = 5000)`
- `Status CloseOdometryStream(int timeout_ms = 5000)`
- `Status SubscribeOdometry(std::function<void(const Odometry&)> callback)`
- `Status UnsubscribeOdometry()`
- `PointCloud2 GetPointCloudMap(int timeout_ms = 5000)`

12.3 Data types

- `SlamMode`: SLAM mode enum.
- `NavMode`: navigation mode enum.
- `Pose3DEuler`: 3D pose with Euler angles.
- `MapImageData`: map image data.
- `MapMetaData`: map metadata.
- `MapInfo`: single map metadata.

- `AllMapInfo`: map collection.
- `LocalizationInfo`: current localization state.
- `NavTarget`: navigation target pose.
- `NavStatusType`: navigation status enum.
- `NavStatus`: navigation task status.
- `Odometry`: odometry data.

CHAPTER 13

Robot Main Control Service

`MagicRobot` is the Python SDK entry point. Use it to initialize resources, connect to the robot, access opened controllers, disconnect, and shut down.

CHAPTER 14

High-Level Motion Control Service

The Python high-level motion controller exposes the same Y1 concepts as the C++ API: WalkId state control, MotionId mode control, and joystick control.

See the *C++ high-level motion page* for the current Y1 diagrams.

CHAPTER 15

Low-Level Motion Control Service

This capability is under development. Public documentation is not available yet.

CHAPTER 16

Sensor Service

This capability is under development. Public documentation is not available yet.

CHAPTER 17

Audio Service

This capability is under development. Public documentation is not available yet.

CHAPTER 18

Monitor Service

The monitor service provides robot state query and subscription capabilities through Python bindings.

CHAPTER 19

SLAM Navigation Service (Python)

`SlamNavController` provides SLAM and navigation control for MagicDog-Y1 in Python. Use it to switch between mapping and localization, manage maps, set navigation targets, read navigation status, and subscribe to odometry.

19.1 Main workflow

1. Initialize the robot and get `SlamNavController`.
2. Call `initialize()` before using SLAM or navigation APIs.
3. Use `activate_slam_mode()` to enter mapping or localization mode.
4. Use mapping APIs to create and save maps, or load an existing map for localization.
5. Use `activate_nav_mode()` and `set_nav_target()` to start navigation.
6. Use status and odometry APIs to monitor navigation.
7. Call `shutdown()` when the controller is no longer needed.

19.2 Controller APIs

- `SlamNavController()`
- `bool initialize()`
- `void shutdown()`

- `Status activate_slam_mode(SlamMode mode, str map_path = "", int timeout_ms = 10000)`
- `Status start_mapping(int timeout_ms = 10000)`
- `Status cancel_mapping(int timeout_ms = 10000)`
- `Status save_map(str map_name, int timeout_ms = 10000)`
- `Status load_map(str map_path, int timeout_ms = 10000)`
- `Status delete_map(str map_path, int timeout_ms = 10000)`
- `str get_map_path()`
- `AllMapInfo get_all_map_info()`
- `Status init_pose(Pose3DEuler pose, int timeout_ms = 10000)`
- `LocalizationInfo get_current_localization_info()`
- `Status activate_nav_mode(NavMode mode, int timeout_ms = 10000)`
- `Status set_nav_target(NavTarget target, int timeout_ms = 10000)`
- `Status pause_nav_task(int timeout_ms = 10000)`
- `Status resume_nav_task(int timeout_ms = 10000)`
- `Status cancel_nav_task(int timeout_ms = 10000)`
- `NavStatus get_nav_task_status()`
- `Status open_odometry_stream(int timeout_ms = 10000)`
- `Status close_odometry_stream(int timeout_ms = 10000)`
- `Status subscribe_odometry(callback)`
- `Status unsubscribe_odometry()`
- `PointCloud2 get_point_cloud_map(int timeout_ms = 10000)`

19.3 Data types

- `SlamMode`: SLAM mode enum.
- `NavMode`: navigation mode enum.
- `Pose3DEuler`: 3D pose with Euler angles.
- `MapImageData`: map image data.
- `MapMetaData`: map metadata.
- `MapInfo`: single map metadata.

- `AllMapInfo`: map collection.
- `LocalizationInfo`: current localization state.
- `NavTarget`: navigation target pose.
- `NavStatusType`: navigation status enum.
- `NavStatus`: navigation task status.
- `Odometry`: odometry data.

CHAPTER 20

High-Level Motion Example

The high-level motion example covers robot initialization, connection, WalkId state switching, MotionId mode switching, and joystick control.

Use the current SDK header files for exact enum names. Different SDK versions may expose different enum identifiers while keeping the same call flow.

CHAPTER 21

Low-Level Motion Example

This capability is under development. Public examples are not available yet.

CHAPTER 22

Sensor Example

This capability is under development. Public examples are not available yet.

CHAPTER 23

Audio Example

This capability is under development. Public examples are not available yet.

CHAPTER 24

Monitor Example

The monitor example is being normalized for the public Y1 documentation. Use the SDK examples as the source of truth until this page is expanded.

CHAPTER 25

FAQ

25.1 Which capabilities are public?

The current documentation opens robot control, high-level motion control, SLAM navigation, and monitor APIs.

25.2 Which capabilities are under development?

Sensor, audio, and low-level motion APIs and examples are under development and are not public yet.

25.3 Does Y1 support wireless development?

Use a wired connection for development. Connect the development computer to the robot switch and configure the communication interface in the `192.168.54.X` subnet. `192.168.54.111` is recommended for the development computer.

25.4 What should I do if an SDK API returns `Deadline Exceeded`?

Some motion or navigation operations can take longer than the default RPC timeout. Set the API's `timeout_ms` parameter based on the expected task duration.

25.5 What should I check if topic subscription does not work?

Confirm that the local network interface is in the `192.168.54.X` subnet and that UDP multicast routing is configured. For example, if the robot-facing interface is `enp0s31f6`, run:

```
sudo ifconfig enp0s31f6 multicast
sudo route add -net 224.0.0.0 netmask 240.0.0.0 dev enp0s31f6
```

25.6 What does `Call of pthread_setschedparam with insufficient privileges!` mean?

The SDK process is running as a regular user without thread-priority permissions. Add this line to `/etc/security/limits.conf`:

```
*    -   rtprio   98
```

25.7 What should I do if I see `Invalid IP address`?

Check whether the app or another SDK client is connected to the robot. Avoid controlling the robot from multiple clients at the same time.

25.8 Why does SLAM navigation not start?

Confirm that the robot has entered localization mode and navigation mode, and that relocalization has succeeded. Before setting a navigation target, read the current localization information and set the target based on the current pose.